

CURSO COMPLETO

SQL



PostgreSQL

MÓDULO 6

VIEWS

VIEWS

VIEWS

O QUE VEREMOS NESTE MÓDULO

MÓDULO 6 VIEWS NO POSTGRESQL

01	Introdução	198
02	CREATE or REPLACE View	201
03	ALTER View	205
04	DROP (IF EXISTS) View	207
05	EXERCÍCIOS	209
06	GABARITO EXPLICATIVO	214

Até aqui vimos como criar diferentes consultas aos bancos de dados. Para isso, aprendemos a utilizar comandos como o SELECT, o GROUP BY, JOINS, etc.

Mas onde foram parar todas esses resultados que criamos? Eles estão em algum lugar?

A resposta é: eles não estão em lugar algum!

Tudo o que fizemos até agora foi apenas visualizar alguns dados das nossas tabelas do banco de dados, nada além disso. Quando executamos um SELECT e logo em seguida executamos outro SELECT, o resultado do primeiro é perdido.

Nenhuma das consultas que fizemos ficou salvo em algum lugar. Inclusive, diversas vezes precisamos criar as mesmas consultas, pois elas se perdem a cada novo SELECT, ou quando fechamos uma consulta e abrimos uma nova.

Existe uma solução para conseguirmos salvar essas *queries* em algum lugar, e esta solução é a **View**.

- Uma View (ou, traduzindo, uma exibição), é uma tabela virtual criada a partir de uma consulta a uma ou mais tabelas (ou até mesmo a outras Views) do banco de dados.
- Ela contém linhas e colunas, assim como uma tabela real. Nela, podemos utilizar comandos como o JOIN, o WHERE, e diversas outras funções.
- As Views sempre mostram resultados atualizados dos dados, ou seja, uma vez criadas, caso haja alterações no banco de dados, elas são atualizadas automaticamente.
- Caso o servidor seja desligado (ou o pgAdmin 4 fechado), a View continua armazenada no sistema.

Através de uma View, conseguimos armazenar uma consulta e acessá-la sempre que precisar, como se fosse uma tabela, com a vantagem de não precisar recriar esse SELECT do zero.

São muitas as vantagens de uma View. Abaixo temos algumas das principais:

Reutilização



Sempre que necessário, podemos consultar aquela View, pois ela fica armazenada no sistema.

Segurança



Ao criar uma View, estamos ocultando linhas ou colunas da tabela original do banco de dados. Desta forma, apenas algumas informações relevantes serão visualizadas na View.

Ganho de tempo



Quando criamos Views, estamos poupando o tempo de recriar vários SELECTs, o que aumenta a produtividade.

CREATE or REPLACE View

201

Vamos a um exemplo bem simples de como criar uma View.

Vamos supor que fizemos uma consulta às colunas product_id, product_name e unit_price da tabela products e queremos armazenar essa consulta em uma View para utilizá-la posteriormente.

Para isso, utilizamos o comando CREATE or REPLACE View:

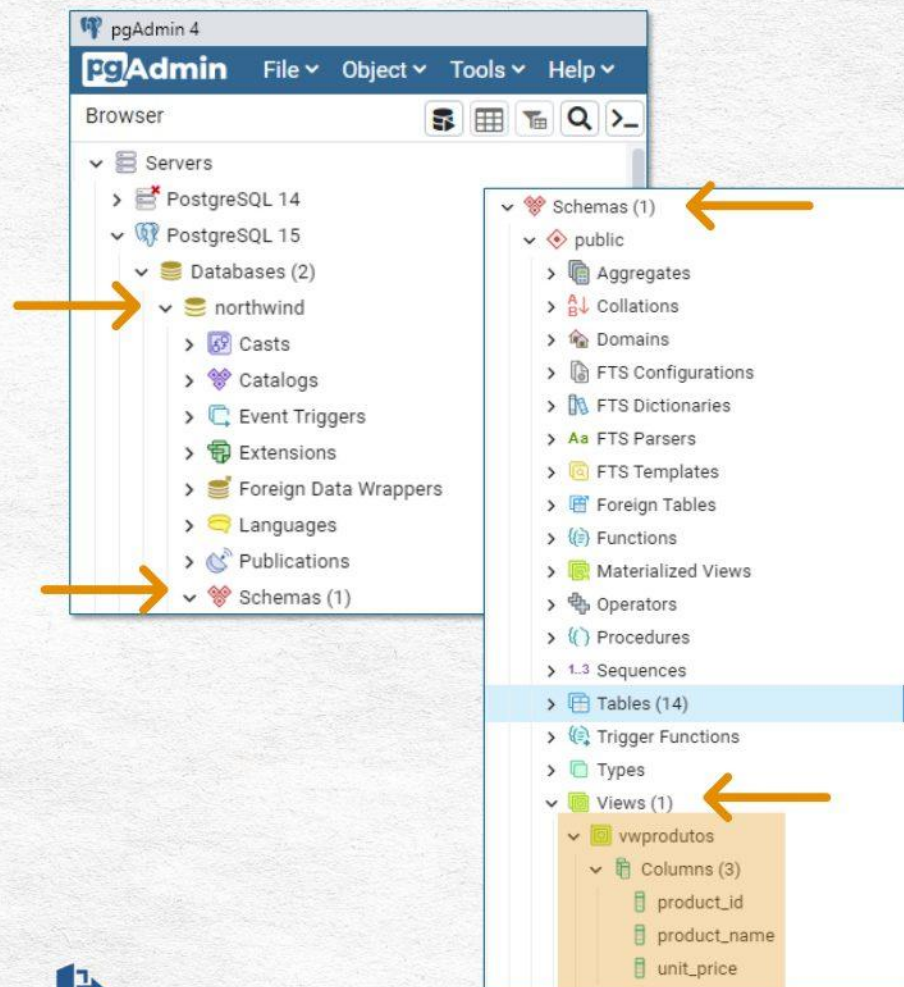
```
CREATE OR REPLACE VIEW vwprodutos AS  
SELECT  
    product_id,  
    product_name,  
    unit_price  
FROM products;
```

Data output	Messages	Notifications
CREATE VIEW		
Query returned successfully in 62 msec.		

CREATE or REPLACE View

202

Uma vez criada, a View ficará armazenada no banco de dados, dentro de Schemas > Views:



CREATE or REPLACE View

203

Para consultar uma VIEW, utilizamos o comando SELECT e informamos o nome da VIEW:

```
SELECT * FROM vwprodutos;
```



Data output Messages Notifications			
	product_id smallint	product_name character varying (40)	unit_price real
1	1	Chai	18
2	2	Chang	19
3	3	Aniseed Syrup	10
4	4	Chef Anton's Cajun Se...	22
5	5	Chef Anton's Gumbo ...	21.35
6	6	Grandma's Boysenber...	25
7	7	Uncle Bob's Organic D...	30
8	8	Northwoods Cranberr...	40
9	9	Mishi Kobe Niku	97
10	10	Ikura	31
11	11	Queso Cabrales	21
Total rows: 77 of 77			Query complete 00:00:00.062

CREATE or REPLACE View

204

Imagine que nos esquecemos de incluir a coluna `units_in_stock` na View, por isso precisamos alterá-la.

Para isso, utilizamos novamente o comando `CREATE or REPLACE View`, acrescentando a coluna desejada no nosso `SELECT`:

```
CREATE OR REPLACE VIEW vwprodutos AS
SELECT
    product_id,
    product_name,
    unit_price,
    units_in_stock
FROM products;
```

Data output Messages Notifications
CREATE VIEW
Query returned successfully in 73 msec.

Observe o resultado com a nova coluna incluída:



Data output Messages Notifications				
	product_id smallint	product_name character varying (40)	unit_price real	units_in_stock smallint
1	1	Chai	18	39
2	2	Chang	19	17
3	3	Aniseed Syrup	10	13
4	4	Chef Anton's Cajun Seasoning	22	53
5	5	Chef Anton's Gumbo Mix	21.35	0
6	6	Grandma's Boysenberry Spread	25	120
Total rows: 77 of 77 Query complete 00:00:00.072				



Se utilizamos os dois comandos no mesmo código (`CREATE` e `REPLACE`), fica mais prático, pois, caso a `VIEW` ainda não exista em nosso banco de dados, ela é criada; se ela já existir, é substituída.

Nomeamos nossa View como “vwprodutos”. E se quisermos alterar para “vw_prod”, como fazemos?

Para isso, existe o comando ALTER VIEW:

```
ALTER VIEW vwprodutos RENAME TO vw_prod;
```

Data output Messages Notifications

ALTER VIEW

Query returned successfully in 97 msec.

Agora, quando quisermos consultá-la, devemos utilizar seu novo nome:

```
SELECT * FROM vw_prod;
```



Data output Messages Notifications					
	product_id	product_name	unit_price	units_in_stock	
	smallint	character varying (40)	real	smallint	
1	1	Chai	18	39	
2	2	Chang	19	17	
3	3	Aniseed Syrup	10	13	
Total rows: 77 of 77 Query complete 00:00:00.092					

Se tentarmos utilizar seu antigo nome, será retornado um erro, pois o sistema entende que aquela View não existe mais:

```
SELECT * FROM vwprodutos;
```



Data output	Messages	Notifications
ERROR: relation "vwprodutos" does not exist LINE 1: SELECT * FROM vwprodutos; ^ SQL state: 42P01 Character: 15		

DROP (IF EXISTS) View

207

E se quisermos excluir a View, como devemos proceder?

Para isso, existem os comandos DROP IF EXISTS VIEW e DROP VIEW.

O DROP IF EXISTS VIEW primeiro verifica se a View existe: se existir, ele procede à exclusão:

```
DROP VIEW IF EXISTS vw_prod;
```



Data output	Messages	Notifications
DROP VIEW		
Query returned successfully in 78 msec.		

Se ele verificar que a View não existe, retorna uma mensagem de alerta na tela, mas o sistema permanece em execução:

```
DROP VIEW IF EXISTS vw_prod;
```



Data output	Messages	Notifications
NOTICE: view "vw_prod" does not exist, skipping DROP VIEW		
Query returned successfully in 46 msec.		

DROP (IF EXISTS) View

208

Neste ponto, o comando DROP IF EXISTS VIEW acaba sendo melhor que o comando DROP VIEW, pois este último não verifica antes se a View existe:

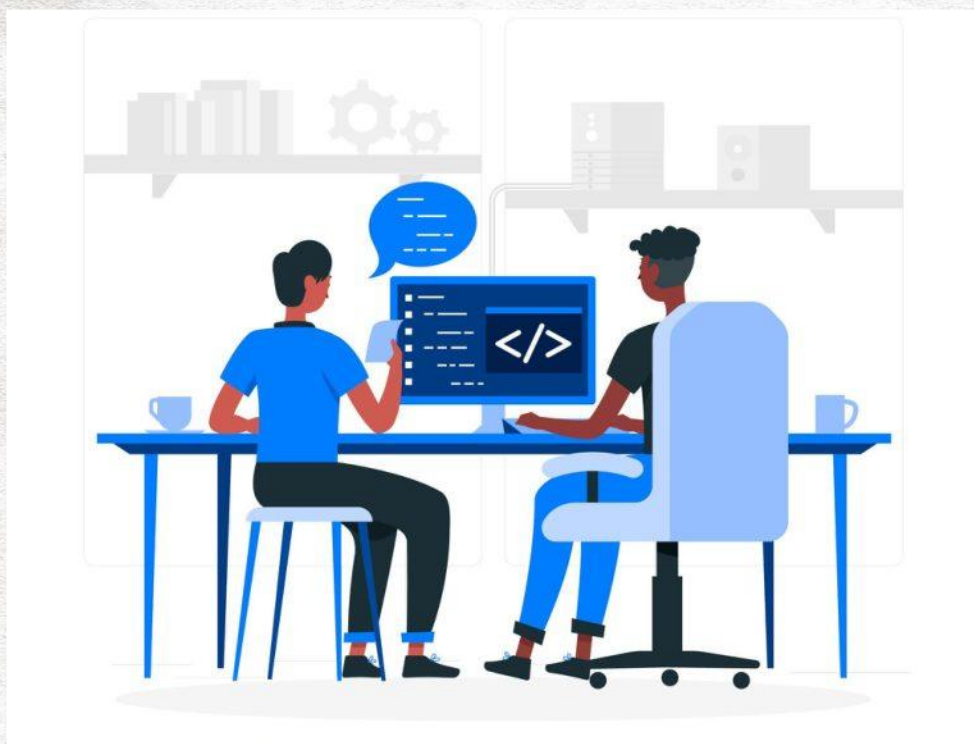
- Se a View existir, ele a exclui;
- Caso a View não exista, é retornado um erro na tela e o sistema é pausado, o que pode ser prejudicial para bancos de dados funcionando em tempo real:

```
DROP VIEW vwprodutos;
```



Data output	Messages	Notifications
ERROR: view "vwprodutos" does not exist SQL state: 42P01		

EXERCÍCIOS



1

Questão 1

A equipe de vendas frequentemente precisa entrar em contato com os clientes para discutir propostas ou resolver problemas relacionados a pedidos. Para facilitar esse processo, é necessário ter acesso rápido às informações de contato de cada cliente.

Crie uma view chamada `vw_clientes_contato` que retorne o ID, o nome da empresa, o nome do contato, a função do contato e o telefone de todos os clientes. (Utilize a tabela `customers`)

2

Questão 2

O setor de logística tem enfrentado desafios para equilibrar os níveis de estoque das diferentes categorias de produtos. Para otimizar a reposição e evitar excessos ou faltas, eles precisam de uma visão consolidada que relacione cada categoria à quantidade total de itens em estoque.

Crie uma view chamada `vw_estoque_por_categoria` para atender a essa necessidade. (Utilize as tabelas `categories` e `products`)

EXERCÍCIOS

3

Questão 3

O gerente comercial está focado em entender o desempenho do mercado na América do Norte (México, EUA e Canadá). Isso inclui monitorar os pedidos realizados por clientes dessa região, identificando padrões de consumo e áreas de potencial crescimento.

Crie uma view chamada `vw_pedidos_america_norte` para retornar o ID e a data do pedido, o nome e o país do cliente, considerando-se somente os países da América do Norte. **(Utilize as tabelas `orders` e `customers`)**

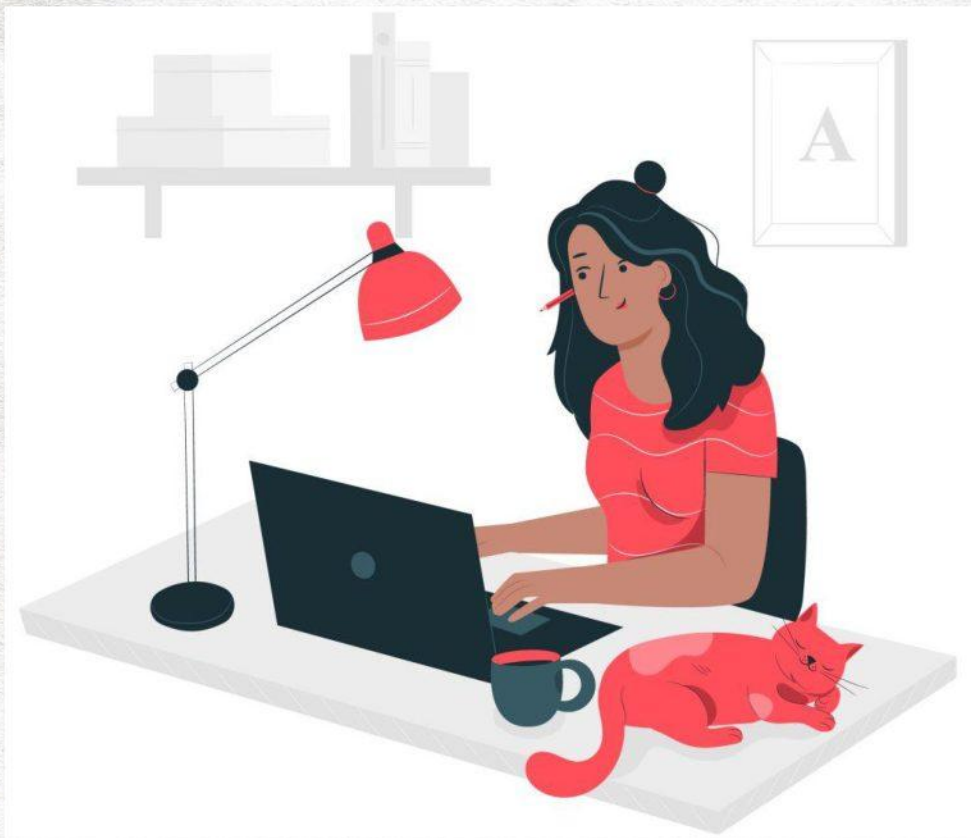
4

Questão 4

O gerente deseja ter acesso rápido aos pedidos mais recentes para acompanhar o fluxo de vendas e identificar tendências ou possíveis gargalos no processo de pedidos.

Crie uma view chamada `vw_ultimos_pedidos` para atender a esse pedido. **(Utilize a tabela `orders`)**

EXERCÍCIOS



5

Questão 5

A equipe de vendas está priorizando clientes que geram alto volume de receita. Por isso, é essencial identificar aqueles cujos pedidos somam uma receita total acima de R\$30.000, para oferecer um atendimento mais dedicado e estratégico.

Crie uma view chamada `vw_clientes_receita_total` para exibir esses clientes e suas respectivas receitas totais (soma de `unit_price * quantity`). (Utilize as tabelas `customers`, `orders` e `orders_details`)

6

Questão 6

O setor de análise está conduzindo uma pesquisa para entender quais categorias de produtos têm maior representatividade no portfólio. Essa informação ajudará a direcionar esforços de marketing e otimizar o mix de produtos oferecidos.

Crie uma view chamada `vw_top5_categorias` para exibir as cinco categorias com maior quantidade de produtos. (Utilize as tabelas `categories` e `products`)

EXERCÍCIOS



7a

Questão 7-a

A gerência quer identificar os mercados mais lucrativos ao redor do mundo. Para isso, é necessário calcular a média de vendas por país, permitindo que os recursos sejam direcionados para os mercados com maior retorno financeiro.

Crie uma view chamada `vw_media_vendas_dos_paises` que mostre os nomes dos países e suas respectivas medias de vendas (média de `quantity * unit_price`). (Utilize as tabelas `orders` e `order_details`)

7b

Questão 7-b

Para facilitar a comunicação entre os setores, o nome da view criada no item A dever ser ajustado para um padrão que reflete melhor seu propósito: `vw_media_vendas_por_pais`.

Faça essa alteração no banco de dados. (Não é necessário utilizar tabelas)

EXERCÍCIOS



8

Questão 8

A diretoria quer analisar os pedidos realizados no ano de 1998 para identificar tendências históricas e avaliar o desempenho da empresa em períodos passados, o que pode ajudar na projeção de vendas futuras.

Crie uma view chamada `vw_pedidos_1998` para listar esses pedidos. (Utilize a tabela `orders`)

9

Questão 9

Exclua todas as views criadas nesta lista de exercícios:

`vw_clientes_contato`
`vw_estoque_por_categoria`
`vw_pedidos_america_norte`
`vw_ultimos_pedidos`
`vw_clientes_receita_total`
`vw_top5_categorias`
`vw_media_vendas_por_pais`
`vw_pedidos_1998`

(Não é necessário utilizar tabelas)

GABARITO EXPLICATIVO

1

Questão 1

A equipe de vendas frequentemente precisa entrar em contato com os clientes para discutir propostas ou resolver problemas relacionados a pedidos. Para facilitar esse processo, é necessário ter acesso rápido às informações de contato de cada cliente.

Crie uma view chamada `vw_clientes_contato` que retorne o ID, o nome da empresa, o nome do contato, a função do contato e o telefone de todos os clientes. (Utilize a tabela `customers`)

Explicação e Resultado

Para criar uma View com o ID, o nome da empresa, o nome do contato, a função do contato e o telefone de todos os clientes, seguimos os passos abaixo:

Utilizamos o comando **CREATE OR REPLACE VIEW** para criar uma nova View chamada `vw_clientes_contato`.

No comando **SELECT** dentro da definição da View, selecionamos as colunas `customer_id`, `company_name`, `contact_name`, `contact_title` e `phone` da tabela **CUSTOMERS**.

Executamos o código para criar a View no banco de dados:

```
CREATE OR REPLACE VIEW vw_clientes_contato AS
SELECT
    customer_id,
    company_name,
    contact_name,
    contact_title,
    phone
FROM customers;
```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 74 msec.

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

1

Questão 1

A equipe de vendas frequentemente precisa entrar em contato com os clientes para discutir propostas ou resolver problemas relacionados a pedidos. Para facilitar esse processo, é necessário ter acesso rápido às informações de contato de cada cliente.

Crie uma view chamada `vw_clientes_contato` que retorne o ID, o nome da empresa, o nome do contato, a função do contato e o telefone de todos os clientes. (Utilize a tabela `customers`)

Explicação e Resultado

Após criar a View, utilizamos o comando `SELECT * FROM vw_clientes_contato` para verificar o conteúdo da View:

```
SELECT * FROM vw_clientes_contato;
```

Dessa forma, retornamos o resultado abaixo, com as informações dos 91 clientes cadastrados:

	customer_id character	company_name character varying (40)	contact_name character varying (30)	contact_title character varying (30)	phone character varying (14)
1	ALFKI	Alfreds Futterkiste	Maria Anders	Sales Representative	030-0074321
2	ANATR	Ana Trujillo Emparedados ...	Ana Trujillo	Owner	(5) 555-4729
3	ANTON	Antonio Moreno Taquería	Antonio Moreno	Owner	(5) 555-3932
4	AROUT	Around the Horn	Thomas Hardy	Sales Representative	(171) 555-7788
5	BERGS	Berglunds snabbköp	Christina Berglund	Order Administrator	0921-12 34 65
Total rows: 91 of 91		Query complete 00:00:00.206			

GABARITO EXPLICATIVO

2

Questão 2

O setor de logística tem enfrentado desafios para equilibrar os níveis de estoque das diferentes categorias de produtos. Para otimizar a reposição e evitar excessos ou faltas, eles precisam de uma visão consolidada que relacione cada categoria à quantidade total de itens em estoque.

Crie uma view chamada `vw_estoque_por_categoria` para atender a essa necessidade. (Utilize as tabelas `categories` e `products`)

Explicação e Resultado

Para criar uma View que mostre a relação entre cada categoria e a quantidade total de itens em estoque, seguimos os seguintes passos:

Utilizamos o comando **CREATE OR REPLACE VIEW** para criar a View chamada `vw_estoque_por_categoria`.

Dentro da definição da View, fazemos um **INNER JOIN** entre as tabelas **CATEGORIES** e **PRODUCTS**, relacionando-as pela coluna `category_id`.

Selecionamos a coluna `category_name` da tabela **CATEGORIES** e utilizamos a função agregada **SUM(p.units_in_stock)** para somar a quantidade total de itens em estoque por categoria.

Agrupamos os resultados pela coluna `category_name` com o comando **GROUP BY**.

Executamos o código para criar a View no banco de dados:

```
CREATE OR REPLACE VIEW vw_estoque_por_categoria AS
SELECT
    c.category_name,
    SUM(p.units_in_stock) AS total_em_estoque
FROM categories c
INNER JOIN products p
    ON c.category_id = p.category_id
GROUP BY c.category_name;
```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 109 msec.

GABARITO EXPLICATIVO

2

Questão 2

O setor de logística tem enfrentado desafios para equilibrar os níveis de estoque das diferentes categorias de produtos. Para otimizar a reposição e evitar excessos ou faltas, eles precisam de uma visão consolidada que relacione cada categoria à quantidade total de itens em estoque.

Crie uma view chamada `vw_estoque_por_categoria` para atender a essa necessidade. (Utilize as tabelas `categories` e `products`)

Explicação e Resultado

Após criar a View, utilizamos o comando `SELECT * FROM vw_estoque_por_categoria` para verificar o conteúdo da View:

```
SELECT * FROM vw_estoque_por_categoria;
```

Dessa forma, retornamos o resultado abaixo, com as informações de estoque por categoria:

	category_name character varying (15) 🔒	total_em_estoque bigint 🔒
1	Beverages	559
2	Produce	100
3	Condiments	507
4	Grains/Cereals	308
5	Meat/Poultry	165
6	Confections	386
7	Dairy Products	393
8	Seafood	701
Total rows: 8 of 8		Query complete 00:00:00.13

GABARITO EXPLICATIVO

3

Questão 3

O gerente comercial está focado em entender o desempenho do mercado na América do Norte (México, EUA e Canadá). Isso inclui monitorar os pedidos realizados por clientes dessa região, identificando padrões de consumo e áreas de potencial crescimento.

Crie uma view chamada `vw_pedidos_america_norte` para retornar o ID e a data do pedido, o nome e o país do cliente, considerando-se somente os países da América do Norte. (Utilize as tabelas `orders` e `customers`)

Explicação e Resultado

Para **criar uma View** que retorna o ID e a data do pedido, o nome e o país do cliente, considerando somente os países da América do Norte, seguimos os seguintes passos:

Utilizamos o comando **CREATE OR REPLACE VIEW** para criar a View chamada `vw_pedidos_america_norte`.

Fazemos um **INNER JOIN** entre as tabelas **ORDERS** e **CUSTOMERS** relacionando-as pela coluna `customer_id`.

Utilizamos a cláusula **WHERE** para filtrar os países desejados: **México, EUA e Canadá**.

Executamos o código para criar a View no banco de dados:

```
CREATE OR REPLACE VIEW vw_pedidos_america_norte AS
SELECT
    o.order_id,
    o.order_date,
    c.company_name,
    c.country
FROM orders o
INNER JOIN customers c
    ON o.customer_id = c.customer_id
WHERE c.country IN ('Mexico', 'USA', 'Canada');
```

Data Output	Messages	Notifications
CREATE VIEW		
Query returned successfully in 72 msec.		

GABARITO EXPLICATIVO

3

Questão 3

O gerente comercial está focado em entender o desempenho do mercado na América do Norte (México, EUA e Canadá). Isso inclui monitorar os pedidos realizados por clientes dessa região, identificando padrões de consumo e áreas de potencial crescimento.

Crie uma view chamada `vw_pedidos_america_norte` para retornar o ID e a data do pedido, o nome e o país do cliente, considerando-se somente os países da América do Norte. (Utilize as tabelas `orders` e `customers`)

Explicação e Resultado

Após criar a View, utilizamos o comando `SELECT * FROM vw_pedidos_america_norte` para verificar o conteúdo da View:

```
SELECT * FROM vw_pedidos_america_norte;
```

Dessa forma, retornamos o resultado abaixo, com os 180 pedidos da América do Norte:

	order_id smallint	order_date date	company_name character varying (40)	country character var
1	10259	1996-07-18	Centro comercial Moctezuma	Mexico
2	10262	1996-07-22	Rattlesnake Canyon Grocery	USA
3	10269	1996-07-31	White Clover Markets	USA
4	10271	1996-08-01	Split Rail Beer & Ale	USA
5	10272	1996-08-02	Rattlesnake Canyon Grocery	USA
6	10276	1996-08-08	Tortuga Restaurante	Mexico
7	10293	1996-08-29	Tortuga Restaurante	Mexico
8	10294	1996-08-30	Rattlesnake Canyon Grocery	USA
Total rows: 180 of 180		Query complete 00:00:00.085		

GABARITO EXPLICATIVO

4

Questão 4

O gerente deseja ter acesso rápido aos pedidos mais recentes para acompanhar o fluxo de vendas e identificar tendências ou possíveis gargalos no processo de pedidos.

Crie uma view chamada `vw_ultimos_pedidos` para atender a esse pedido. (Utilize a tabela `orders`)

Explicação e Resultado

Para criar uma View que retorna os 5 pedidos mais recentes, seguimos os seguintes passos: Utilizamos o comando **CREATE OR REPLACE VIEW** para criar a View chamada `vw_ultimos_pedidos`.

Selecionamos todos os dados da tabela **ORDERS** e ordenamos pelo campo `order_date` de forma decrescente, utilizando **ORDER BY order_date DESC**.

Limitamos o resultado aos 5 pedidos mais recentes com o comando **LIMIT 5**.

Executamos o código para criar a View no banco de dados:

```
CREATE OR REPLACE VIEW vw_ultimos_pedidos AS  
SELECT  
    *  
FROM orders  
ORDER BY order_date DESC  
LIMIT 5;
```

Data Output	Messages	Notifications
CREATE VIEW		
Query returned successfully in 118 msec.		

CONTINUA NA PRÓXIMA PÁGINA

GABARITO EXPLICATIVO

4

Questão 4

O gerente deseja ter acesso rápido aos pedidos mais recentes para acompanhar o fluxo de vendas e identificar tendências ou possíveis gargalos no processo de pedidos.

Crie uma view chamada `vw_ultimos_pedidos` para atender a esse pedido. (Utilize a tabela `orders`)

Explicação e Resultado

Após criar a View, utilizamos o comando `SELECT * FROM vw_ultimos_pedidos` para verificar o conteúdo da View:

```
SELECT * FROM vw_ultimos_pedidos;
```

Dessa forma, retornamos o resultado abaixo, com os 5 pedidos mais recentes:

	order_id smallint	customer_id character	employee_id smallint	order_date date	required_date date	shipped_date date	ship_via smallint	freight real	ship_name character varying (40)	ship_address character varying (60)	ship_city character varyin	ship_region character var	ship_postal_code character varying (	ship_country character varyin
1	11076	BONAP	4	1998-05-06	1998-06-03	[null]	2	38.28	Bon app'	12, rue des Bouchers	Marseille	[null]	13008	France
2	11077	RATTC	1	1998-05-06	1998-06-03	[null]	2	8.53	Rattlesnake Canyon Grocery	2817 Milton Dr.	Albuquerque	NM	87110	USA
3	11074	SIMOB	7	1998-05-06	1998-06-03	[null]	2	18.44	Simons bistro	Vinbæltet 34	Kobenhavn	[null]	1734	Denmark
4	11075	RICSU	8	1998-05-06	1998-06-03	[null]	2	6.19	Richter Supermarkt	Starenweg 5	Genève	[null]	1204	Switzerland
5	11073	PERIC	2	1998-05-05	1998-06-02	[null]	2	24.95	Pericles Comidas clásicas	Calle Dr. Jorge Cash 321	México D.F.	[null]	05033	Mexico
Total rows: 5 of 5 Query complete 00:00:00.165														

GABARITO EXPLICATIVO

5

Questão 5

A equipe de vendas está priorizando clientes que geram alto volume de receita. Por isso, é essencial identificar aqueles cujos pedidos somam uma receita total acima de R\$30.000, para oferecer um atendimento mais dedicado e estratégico.

Crie uma view chamada `vw_clientes_receita_total` para exibir esses clientes e suas respectivas receitas totais (soma de `unit_price * quantity`). (Utilize as tabelas `customers`, `orders` e `orders_details`)

Explicação e Resultado

Para criar uma View que retorna os clientes que geram receitas totais acima de R\$30.000, seguimos os seguintes passos:

Utilizamos o comando **CREATE OR REPLACE VIEW** para criar a View chamada `vw_clientes_receita_total`.

Fazemos dois **INNER JOINS**: um entre as tabelas **CUSTOMERS** e **ORDERS**, relacionando-as pela coluna `customer_id`; outro entre as tabelas **ORDERS** e **ORDER_DETAILS**, relacionando-as pela coluna `order_id`.

Utilizamos a função **SUM(od.unit_price * od.quantity)** para calcular a `receita_total` de cada cliente, **agrupando** pelas colunas `customer_id` e `company_name`, aplicando, em seguida, a cláusula **HAVING** para filtrar os clientes cuja receita gerada seja superior a R\$30.000.

Executamos o código para criar a View no banco de dados:

```
CREATE OR REPLACE VIEW vw_clientes_receita_total AS
SELECT
    c.customer_id,
    c.company_name,
    SUM(od.unit_price * od.quantity) AS receita_total
FROM customers c
INNER JOIN orders o
    ON c.customer_id = o.customer_id
INNER JOIN order_details od
    ON o.order_id = od.order_id
GROUP BY c.customer_id, c.company_name
HAVING SUM(od.unit_price * od.quantity) > 30000;
```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 89 msec.

GABARITO EXPLICATIVO

5

Questão 5

A equipe de vendas está priorizando clientes que geram alto volume de receita. Por isso, é essencial identificar aqueles cujos pedidos somam uma receita total acima de R\$30.000, para oferecer um atendimento mais dedicado e estratégico.

Crie uma view chamada `vw_clientes_receita_total` para exibir esses clientes e suas respectivas receitas totais (soma de `unit_price * quantity`). (Utilize as tabelas `customers`, `orders` e `orders_details`)

Explicação e Resultado

Após criar a View, utilizamos o comando `SELECT * FROM vw_clientes_receita_total` para verificar o conteúdo da View:

```
SELECT * FROM vw_clientes_receita_total;
```

Dessa forma, retornamos o resultado abaixo, com os clientes e suas respectivas receitas totais:

	customer_id character	company_name character varying (40)	receita_total double precision
1	QUEEN	Queen Cozinha	30226.10017967224
2	RATTC	Rattlesnake Canyon Grocery	52245.90034675598
3	QUICK	QUICK-Stop	117483.390147686
4	HUNGO	Hungry Owl All-Night Grocers	57317.39016246796
5	KOENE	Königlich Essen	31745.749893188477
6	MEREP	Mère Paillard	32203.900234222412
7	HANAR	Hanari Carnes	34101.149973869324
8	FOLKO	Folk och få HB	32555.55001926422
9	SAVEA	Save-a-lot Markets	115673.38964271545
10	ERNSH	Ernst Handel	113236.67978191376
Total rows: 10 of 10		Query complete 00:00:00.094	

GABARITO EXPLICATIVO

6

Questão 6

O setor de análise está conduzindo uma pesquisa para entender quais categorias de produtos têm maior representatividade no portfólio. Essa informação ajudará a direcionar esforços de marketing e otimizar o mix de produtos oferecidos.

Crie uma view chamada `vw_top5_categorias` para exibir as cinco categorias com maior quantidade de produtos. (Utilize as tabelas `categories` e `products`)

Explicação e Resultado

Para criar uma View que retorna as 5 categorias com maior quantidade de produtos, seguimos os seguintes passos:

Utilizamos o comando **CREATE OR REPLACE VIEW** para criar a View chamada `vw_top5_categorias`.

Fazemos um **INNER JOIN** entre as tabelas **CATEGORIES** e **PRODUCTS**, relacionando-as pela coluna `category_id`.

Utilizamos a função **COUNT(p.product_id)** para contar o número de produtos por categoria (**GROUP BY c.category_name**) e ordenamos os resultados pela quantidade de produtos de forma decrescente, utilizando **ORDER BY total_produtos DESC**.

Limitamos o resultado às 5 categorias com mais produtos com o comando **LIMIT 5**.

Executamos o código para criar a View no banco de dados:

```
CREATE OR REPLACE VIEW vw_top5_categorias AS
SELECT
    c.category_name,
    COUNT(p.product_id) AS total_produtos
FROM categories c
INNER JOIN products p
    ON c.category_id = p.category_id
GROUP BY c.category_name
ORDER BY total_produtos DESC
LIMIT 5;
```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 78 msec.

GABARITO EXPLICATIVO

6

Questão 6

O setor de análise está conduzindo uma pesquisa para entender quais categorias de produtos têm maior representatividade no portfólio. Essa informação ajudará a direcionar esforços de marketing e otimizar o mix de produtos oferecidos.

Crie uma view chamada `vw_top5_categorias` para exibir as cinco categorias com maior quantidade de produtos. (Utilize as tabelas `categories` e `products`)

Explicação e Resultado

Após criar a View, utilizamos o comando `SELECT * FROM vw_top5_categorias` para verificar o conteúdo da View:

```
SELECT * FROM vw_top5_categorias;
```

Dessa forma, retornamos o resultado abaixo, com as 5 categorias com maior quantidade de produtos:

	category_name character varying (15) 🔒	total_produtos bigint 🔒
1	Confections	13
2	Condiments	12
3	Beverages	12
4	Seafood	12
5	Dairy Products	10
Total rows: 5 of 5		Query complete 00:00:00.

GABARITO EXPLICATIVO

7a

Questão 7-a

A gerência quer identificar os mercados mais lucrativos ao redor do mundo. Para isso, é necessário calcular a média de vendas por país, permitindo que os recursos sejam direcionados para os mercados com maior retorno financeiro.

Crie uma view chamada `vw_media_vendas_dos_paises` que mostre os nomes dos países e suas respectivas medias de vendas (média de `quantity * unit_price`). (Utilize as tabelas `orders` e `order_details`)

Explicação e Resultado

Para criar uma View que retorna a média de vendas por país, seguimos os seguintes passos: Utilizamos o comando **CREATE OR REPLACE VIEW** para criar a View chamada `vw_media_vendas_dos_paises`.

Fazemos um **INNER JOIN** entre as tabelas **ORDERS** e **ORDER_DETAILS**, relacionando-as pela coluna `order_id`.

Utilizamos a função **AVG(od.quantity * od.unit_price)** para calcular a `media_vendas` por país, agrupando os resultados por país com **GROUP BY o.ship_country**.

Ordenamos os resultados pela média de vendas de forma decrescente com **ORDER BY media_vendas DESC**.

Executamos o código para criar a View no banco de dados:

```
CREATE OR REPLACE VIEW vw_media_vendas_dos_paises AS
SELECT
    o.ship_country,
    AVG(od.quantity * od.unit_price) AS media_vendas
FROM orders o
INNER JOIN order_details od
    ON o.order_id = od.order_id
GROUP BY o.ship_country
ORDER BY media_vendas DESC;
```

Data Output	Messages	Notifications
CREATE VIEW		
Query returned successfully in 76 msec.		

GABARITO EXPLICATIVO

7a

Questão 7-a

A gerência quer identificar os mercados mais lucrativos ao redor do mundo. Para isso, é necessário calcular a média de vendas por país, permitindo que os recursos sejam direcionados para os mercados com maior retorno financeiro.

Crie uma view chamada `vw_media_vendas_dos_paises` que mostre os nomes dos países e suas respectivas medias de vendas (média de `quantity * unit_price`). (Utilize as tabelas `orders` e `order_details`)

Explicação e Resultado

Após criar a View, utilizamos o comando `SELECT * FROM vw_media_vendas_dos_paises` para verificar o conteúdo da View:

```
SELECT * FROM vw_media_vendas_dos_paises;
```

Dessa forma, retornamos o resultado abaixo, com as médias de vendas por país dos 21 países cadastrados:

	ship_country character varying (15)	media_vendas double precision
1	Austria	1115.9730389328004
2	Ireland	1042.1343665903264
3	Denmark	756.1358697207078
4	USA	748.7698295929215
5	Germany	745.8555791726926
6	Canada	737.7880024973551
7	Switzerland	633.067307646458
8	Belgium	627.4103628737586
Total rows: 21 of 21		Query complete 00:00:00.07

GABARITO EXPLICATIVO

7b

Questão 7-b

Para facilitar a comunicação entre os setores, o nome da view criada no item A deve ser ajustado para um padrão que reflète melhor seu propósito: `vw_media_vendas_por_pais`.

Faça essa alteração no banco de dados. (Não é necessário utilizar tabelas)

Explicação e Resultado

Para alterar o nome da View criada no exercício anterior, seguimos os seguintes passos:

Utilizamos o comando **ALTER VIEW** para renomear a View de `vw_media_vendas_dos_paises` para `vw_media_vendas_por_pais`.

Executamos o código para realizar a alteração:

```
ALTER VIEW vw_media_vendas_dos_paises  
RENAME TO vw_media_vendas_por_pais;
```

Data Output	Messages	Notifications
ALTER VIEW		
Query returned successfully in 56 msec.		

Após a alteração, utilizamos o comando **SELECT * FROM** `vw_media_vendas_por_pais` para verificar o conteúdo da View com o novo nome:

```
SELECT * FROM vw_media_vendas_por_pais;
```


GABARITO EXPLICATIVO

7b

Questão 7-b

Para facilitar a comunicação entre os setores, o nome da view criada no item A deve ser ajustado para um padrão que reflète melhor seu propósito: `vw_media_vendas_por_pais`.

Faça essa alteração no banco de dados. (Não é necessário utilizar tabelas)

Explicação e Resultado

Dessa forma, verificamos que a View foi renomeada com sucesso e retornou o resultado esperado (as médias de vendas por país dos 21 países cadastrados):

	ship_country character varying (15) 🔒	media_vendas double precision 🔒
1	Austria	1115.9730389328004
2	Ireland	1042.1343665903264
3	Denmark	756.1358697207078
4	USA	748.7698295929215
5	Germany	745.8555791726926
6	Canada	737.7880024973551
7	Switzerland	633.067307646458
8	Belgium	627.4103628737586
Total rows: 21 of 21		Query complete 00:00:00.07

GABARITO EXPLICATIVO

8

Questão 8

A diretoria quer analisar os pedidos realizados no ano de 1998 para identificar tendências históricas e avaliar o desempenho da empresa em períodos passados, o que pode ajudar na projeção de vendas futuras.

Crie uma view chamada `vw_pedidos_1998` para listar esses pedidos. (Utilize a tabela `orders`)

Explicação e Resultado

Para criar uma View que retorna os pedidos realizados no ano de 1998, seguimos os seguintes passos:

Utilizamos o comando `CREATE OR REPLACE VIEW` para criar a View chamada `vw_pedidos_1998`.

Selecionamos os campos `order_id`, `customer_id` e `order_date` da tabela `ORDERS` e aplicamos a cláusula `WHERE` para filtrar os pedidos realizados entre 1º de janeiro e 31 de dezembro de 1998.

Executamos o código para criar a View no banco de dados:

```
CREATE OR REPLACE VIEW vw_pedidos_1998 AS
SELECT
    order_id,
    customer_id,
    order_date
FROM orders
WHERE order_date BETWEEN '1998-01-01' AND '1998-12-31';
```

Data Output Messages Notifications

CREATE VIEW

Query returned successfully in 65 msec.

GABARITO EXPLICATIVO

8

Questão 8

A diretoria quer analisar os pedidos realizados no ano de 1998 para identificar tendências históricas e avaliar o desempenho da empresa em períodos passados, o que pode ajudar na projeção de vendas futuras.

Crie uma view chamada `vw_pedidos_1998` para listar esses pedidos. (Utilize a tabela `orders`)

Explicação e Resultado

Após criar a View, utilizamos o comando `SELECT * FROM vw_pedidos_1998` para verificar o conteúdo da View:

```
SELECT * FROM vw_pedidos_1998;
```

Dessa forma, retornamos o resultado abaixo, com os 270 pedidos de 1998:

	order_id smallint 🔒	customer_id character 🔒	order_date date 🔒
1	10808	OLDWO	1998-01-01
2	10809	WELLI	1998-01-01
3	10810	LAUGB	1998-01-01
4	10811	LINOD	1998-01-02
5	10812	REGGC	1998-01-02
6	10813	RICAR	1998-01-05
7	10814	VICTE	1998-01-05
8	10815	SAVEA	1998-01-05
Total rows: 270 of 270			Query complete 00:00

GABARITO EXPLICATIVO

9

Questão 9

Exclua todas as views criadas nesta lista de exercícios:

vw_clientes_contato
vw_estoque_por_categoria
vw_pedidos_america_norte
vw_ultimos_pedidos
vw_clientes_receita_total
vw_top5_categorias
vw_media_vendas_por_pais
vw_pedidos_1998

(Não é necessário utilizar tabelas)

Explicação e Resultado

Para excluir todas as views criadas, utilizamos o comando **DROP VIEW IF EXISTS** para cada uma das views. Executando cada código, veremos que todas as views são excluídas com sucesso:

```
DROP VIEW IF EXISTS vw_clientes_contato;
```

```
DROP VIEW
```

```
Query returned successfully in 68 msec.
```

```
DROP VIEW IF EXISTS vw_estoque_por_categoria;
```

```
DROP VIEW
```

```
Query returned successfully in 75 msec.
```

```
DROP VIEW IF EXISTS vw_pedidos_america_norte;
```

```
DROP VIEW
```

```
Query returned successfully in 67 msec.
```

```
DROP VIEW IF EXISTS vw_ultimos_pedidos;
```

```
DROP VIEW
```

```
Query returned successfully in 71 msec.
```

```
DROP VIEW IF EXISTS vw_clientes_receita_total;
```

```
DROP VIEW
```

```
Query returned successfully in 63 msec.
```

```
DROP VIEW IF EXISTS vw_top5_categorias;
```

```
DROP VIEW
```

```
Query returned successfully in 83 msec.
```

```
DROP VIEW IF EXISTS vw_media_vendas_por_pais;
```

```
DROP VIEW
```

```
Query returned successfully in 66 msec.
```

```
DROP VIEW IF EXISTS vw_pedidos_1998;
```

```
DROP VIEW
```

```
Query returned successfully in 72 msec.
```


SQL

PostgreSQL
Apostila completa